CDC & RDC

This is a practice assignment to help you get hands-on experience with checking and fixing CDC/RDC violations. It's completely optional and won't be graded, just a chance to sharpen your skills!

The solution will be posted later, so feel free to review it once it's available. If you have any questions, don't hesitate to reach out.

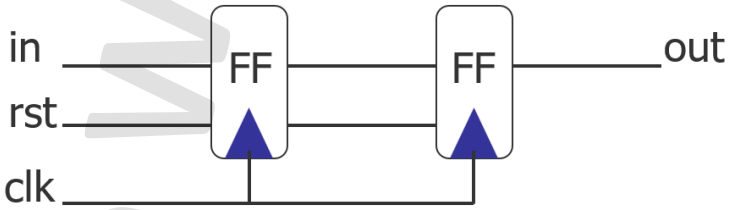
1) Create a Verilog design for a Two-DFF synchronizer.

Parameters:

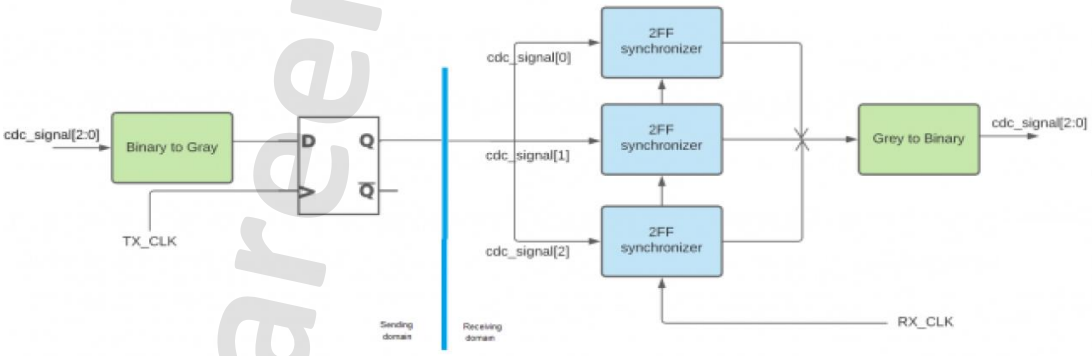
- WIDTH: Determines the width of the input D and Q for the flipflops (Default: 1)
- RSTTYPE: Determines whether the reset of the synchronizer is synchronous (SYNC) or asynchronous (ASYNC) (Default: SYNC)

Ports:

- clk
- rst
- in (width determined by parameter WIDTH)
- out (width determined by parameter WIDTH)



2) Create a Verilog design for a gray encoded synchronizer



Instantiate the two-DFF synchronizer done in question 1. Create binary to gray and gray to binary modules and instantiate them also in the gray encoded synchronizer design.

Kareem Waseem

Parameters:

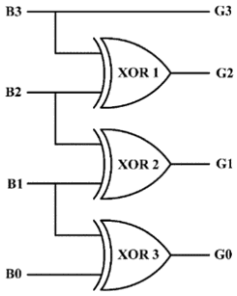
- WIDTH: Determines the width of the inputs tx_data and rx_data (Default: 8)
- RSTTYPE: Determines whether the reset of the registers are synchronous (SYNC) or asynchronous (ASYNC) (Default: SYNC)

Ports:

- tx_data (Input: width determined by parameter WIDTH)
- rx_data (output: width determined by parameter WIDTH)
- tx_clk (input)
- rx_clk (input)
- rst (input)

Notes for the conversion:

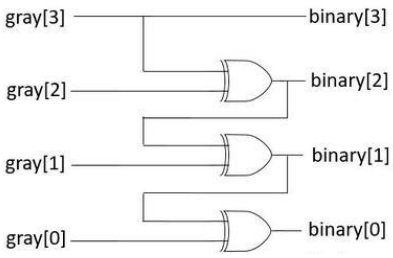
Given a binary number B[n-1:0], the conversion to Gray code G[n-1:0] is computed as:



- G[n-1] = B[n-1] (The MSB stays the same)
- For all other bits (from n-2 to 0):
    - G[i] = B[i+1] ^ B[i]

Given a Gray code input G[n-1:0], the Binary output B[n-1:0] is calculated as:

- B[n-1] = G[n-1] (The MSB stays the same)
- For all other bits (from n-2 to 0):
    - B[i] = B[i+1] ^ G[i] for i from n-2 down to 0

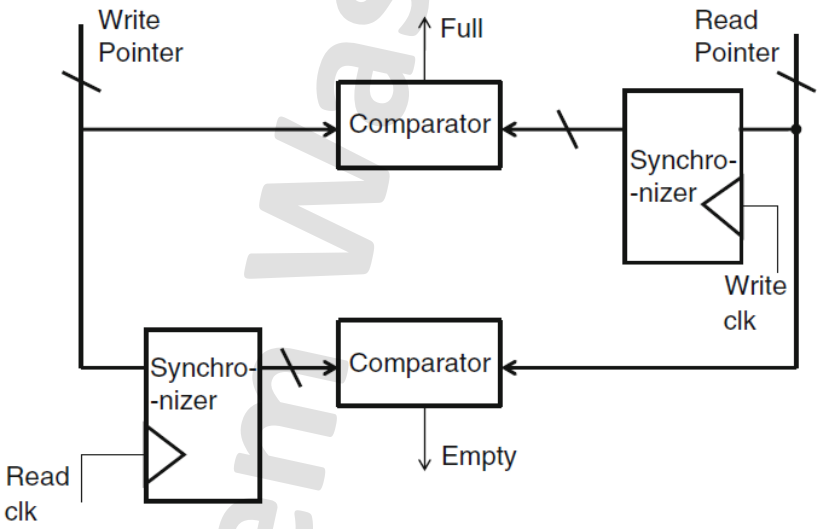


Use Generate-for to implement the conversions above.

Kareem Waseem

3) Modify the FIFO design done in the last assignment. Since writing to the FIFO occurs on clk_a and reading occurs on clk_b, metastability can occur. Therefore, the full flag and empty flag logic must be implemented correctly using proper synchronization.

- The full flag is determined in the write clock domain (clk_a), so the read pointer must be synchronized into the clk_a domain before being used in the full flag logic.
- Similarly, the empty flag is determined in the read clock domain (clk_b), so the write pointer must be synchronized into the clk_b domain before being used in the empty flag logic.
- Instantiate the gray-encoded synchronizer implemented in question 2 in the FIFO and modify the empty and full flag to use the synchronizer outputs



Run Questa CDC to make sure there are no clock domain crossing violations in the FIFO.

4) Modify the DSP48A1 (project 1) to avoid RDC violations in case the DSP block registers are using asynchronous reset. To prevent this, do the following modifications:

- Instantiate the synchronizer (from Question 1) between the appropriate registers in the DSP block to ensure safe reset domain transitions.
- Set the RSTTYPE parameter of the DSP block to "ASYNC" to verify the asynchronous reset behavior.
- Use generate-if to conditionally instantiate the synchronizers only when RSTTYPE is set to "ASYNC".

Run Questa RDC to make sure there are no reset domain crossing violations in the DSP block.

Deliverables:

1) The assignment should be submitted as a PDF file with this format <your_name>_Assignment6 for example Kareem_Waseem_Assignment6
2) Snippets of the RTL codes
3) Snippets from Questa CDC and Questa RDC showing no violations

Kareem Waseem